Universidade Eduardo Mondlane
Faculdade de Engenharia
Departamento de Engenharia Electrotécnica
Curso de Engenharia Informática

INTELIGÊNCIA ARTIFICIAL I

P6 — "Engarrafamento na Baixa" (Rush Hour generalizado)

RELATÓRIO DO PROJECTO PRÁTICO

GRUPO 06 — Laboral

Estudantes

- Falaque, Dalton
- Nhabinde, Alex
- Nhabomba, Lino
- Quive, Alexandre
- Ricardo

Docentes

- Engª Roxan Cadir
- Eng. Ruben Manhiça

Maputo, 18 de Novembro de 2026



Universidade Eduardo Mondlane
Faculdade de Engenharia
Departamento de Engenharia Electrotécnica
Curso de Engenharia Informática

INTELIGÊNCIA ARTIFICIAL I

P6 — “Engarrafamento na Baixa” (Rush Hour generalizado)

RELATÓRIO DO PROJECTO PRÁTICO

GRUPO 06 — Laboral

Estudantes

- Falaque, Dalton
- Nhabinde, Alex
- Nhabomba, Lino
- Quive, Alexandre
- Ricardo

Docentes

- Engª Roxan Cadir
- Eng. Ruben Manhiça

Maputo, 18 de Novembro de 2026

RESUMO

Este relatório formula como problema de busca o P6, “Engarrafamento na Baixa”, uma generalização do puzzle Rush Hour em que o chapa X tem de sair de um parque congestionado pela saída à direita, com tabuleiros M×M até 8×8, obstáculos fixos e dois critérios de optimização: número de movimentos e custo ponderado (comprimento × casas). Implementámos de raiz BFS, DFS, IDS, Custo Uniforme, Gulosa, A* e IDA* com tabela de transposição sobre uma interface genérica, quatro heurísticas (h1–h4) e um gerador de instâncias por busca para trás com a semente 202606. Na instância base, com 1079 estados alcançáveis, o óptimo é de 8 movimentos e custo ponderado 40, e os dois critérios coincidem; o gerador encontrou duas instâncias em que nenhuma solução de mínimo de movimentos é óptima no custo ponderado. Provámos que a dica h2 não é admissível em movimentos (contra-exemplo com h2 = 4 e h* = 3), mas é admissível no custo ponderado. Nas instâncias médias, o A* com h4 expandiu em média 38,7 % menos nós do que o UCS no custo ponderado, e nas pequenas a tabela de transposição reduziu os nós do IDA* em 57,2 %. Concluímos que, para este problema, o A* com uma heurística consistente derivada de um problema relaxado é a melhor escolha, que DFS e IDS só são viáveis em instâncias pequenas e que a dica h2 pode devolver soluções não óptimas (observado numa instância grande).

Palavras-chave: Inteligência Artificial; Busca no espaço de estados; A*; Heurísticas admissíveis; Rush Hour; IDA*; Tabela de transposição; Gerador de instâncias

ABREVIATURAS

A*	Algoritmo A estrela
b*	Factor de ramificação efectivo
BFS	Breadth-First Search — Busca em Largura
DFS	Depth-First Search — Busca em Profundidade
g(n)	Custo do caminho desde o estado inicial até ao nó n
h(n)	Função heurística: estimativa do custo de n até ao objectivo
h*(n)	Custo óptimo real de n até ao objectivo
IA I	Inteligência Artificial I
IDA*	Iterative Deepening A* — A* com aprofundamento iterativo
IDS	Iterative Deepening Search — Aprofundamento Iterativo
TT	Tabela de transposição
UCS	Uniform-Cost Search — Busca de Custo Uniforme

ÍNDICE DE FIGURAS

Figura 1 – Arquitectura do programa.....10
Figura 2 – Nós expandidos em função do tamanho da instância (escala logarítmica), critério movimentos.....17
Figura 3 – Nós expandidos em função do tamanho da instância (escala logarítmica), critério ponderado.....17
Figura 4 – Tempo de execução em função do tamanho da instância (escala logarítmica), critério ponderado.....18
Figura 5 – Custo obtido em função do custo óptimo, critério ponderado.....18
Figura 6 – Nós expandidos pelo IDA* simples e pelo IDA* com tabela de transposição.....19

ÍNDICE DE TABELAS

Tabela 1 – Propriedades das estratégias de busca.....3
Tabela 2 – Acções do problema.....6
Tabela 3 – Comparação das heurísticas no A*.....9
Tabela 4 – Resultados médios por algoritmo — instâncias pequenas, critério movimentos…15
Tabela 5 – Resultados médios por algoritmo — instâncias pequenas, critério ponderado.....15
Tabela 6 – Resultados médios por algoritmo — instâncias médias, critério movimentos.....15
Tabela 7 – Resultados médios por algoritmo — instâncias médias, critério ponderado.....15
Tabela 8 – Resultados médios por algoritmo — instâncias grandes, critério movimentos.....16
Tabela 9 – Resultados médios por algoritmo — instâncias grandes, critério ponderado.....16
Tabela 10 – Instâncias em que os dois critérios de optimização divergem.....21
Tabela 11 – Uso de ferramentas de IA.....24
Tabela 12 – Contribuição individual.....24

ÍNDICE

1 INTRODUÇÃO....1

1.1 Contextualização do problema....1

1.2 Objectivos....1

1.2.1 Objectivo geral....1

1.2.2 Objectivos específicos....1

1.3 Metodologia de trabalho do grupo....2

1.4 Estrutura do relatório....2

2 REVISÃO TEÓRICA....2

2.1 Problemas de busca e espaço de estados....2

2.2 Estratégias de busca não informada....3

2.3 Estratégias de busca informada e heurísticas....3

2.4 Algoritmo avançado atribuído....3

2.5 Abordagens alternativas e trabalhos relacionados....4

3 FORMULAÇÃO DO PROBLEMA DE BUSCA....5

3.1 Descrição do problema e pressupostos....5

3.2 Representação do estado....5

3.3 Estado inicial....6

3.4 Acções e modelo de transição....6

3.5 Função de custo....6

3.6 Teste de objectivo....6

3.7 Dimensão do espaço de estados....7

4 HEURÍSTICAS....7

4.1 Definição das heurísticas....7

4.2 Prova de admissibilidade....8

4.3 Prova de consistência....8

4.4 Verificação empírica e dominância....9

5 IMPLEMENTAÇÃO....10

5.1 Arquitectura do código.....10
5.2 Estruturas de dados.....11
5.3 Algoritmos implementados.....11
5.4 Gerador de instâncias.....12
5.5 Testes automáticos.....13
5.6 Interface e instruções de execução.....13
6 EXPERIÊNCIAS E RESULTADOS.....14
6.1 Desenho experimental.....14
6.2 Resultados.....14
6.3 Análise e discussão.....19
6.4 Resposta à pergunta de análise.....21
7 CONCLUSÕES.....22
7.1 Síntese.....22
7.2 Limitações.....22
7.3 Trabalho futuro.....23
8 REFERÊNCIAS BIBLIOGRÁFICAS.....23
9 ANEXO A — DECLARAÇÃO DE USO DE FERRAMENTAS DE IA.....24
10 ANEXO B — CONTRIBUIÇÃO DOS MEMBROS.....24
11 ANEXO C — CÓDIGO-FONTE RELEVANTE.....25

1 INTRODUÇÃO

1.1 Contextualização do problema

Na Baixa de Maputo, os parques de estacionamento junto aos terminais de chapas enchem-se nas horas de ponta. Quando um chapa precisa de sair, é frequente ter de pedir a vários condutores que desloquem os seus veículos, numa ordem que nem sempre é evidente: mover primeiro o veículo errado pode bloquear a saída ainda mais. O problema atribuído ao Grupo 06 (P6) modela esta situação como uma generalização do puzzle *Rush Hour*: num parque representado por uma grelha, cada veículo ocupa duas casas (carros) ou três (camiões), só se desloca na sua orientação e não pode atravessar outros veículos nem obstáculos fixos, como um carro da polícia estacionado. O objectivo é tirar o chapa X pela saída à direita da linha 3.

O problema é difícil porque o número de configurações cresce exponencialmente com o número de veículos e o *Rush Hour* generalizado é PSPACE-completo (Flake & Baum, 2002). Resolver bem significa encontrar a sequência com o menor número de movimentos (menos pedidos aos condutores) ou com o menor esforço total, se mover um camião custar mais do que mover um carro. Estes dois critérios nem sempre levam à mesma solução, e o grupo mostra-o experimentalmente.

1.2 Objectivos

1.2.1 Objectivo geral

Modelar o problema P6 como problema de busca no espaço de estados e comparar, teórica e experimentalmente, sete estratégias de busca na sua resolução.

1.2.2 Objectivos específicos

1. Formular o problema (estado, acções, transição, custo e objectivo) com uma representação compacta, imutável e *hashable*, para os dois critérios de optimização;
2. Implementar de raiz BFS, DFS, IDS, Custo Uniforme, Gulosa, A* e IDA* com tabela de transposição sobre uma interface genérica;
3. Propor heurísticas, provar (ou refutar) a sua admissibilidade e consistência e verificá-las automaticamente em todos os estados de instâncias pequenas;
4. Construir um gerador reprodutível (semente 202606) de puzzles com solução óptima certificada;
5. Comparar os algoritmos em três níveis de instâncias e interpretar os resultados à luz da teoria.

1.3 Metodologia de trabalho do grupo

O grupo organizou o trabalho num sprint de seis dias (9 a 14 de Novembro de 2026), com um quadro de 35 tarefas (S01–S35) e um responsável por tarefa, documentado em `outputs/Grupo_06_Sprint_Scrum_6_Dias.pdf` e `outputs/Grupo_06_Quadro_Sprint.xlsx`. O código está num repositório Git partilhado (`Alexandros-o-megas/Projecto-IAI`), com um ramo por tarefa (por exemplo `feature/lino/S02-S03-S09-S10`) integrado na `main` por *pull request* após revisão de outro membro. Primeiro fechou-se o contrato do domínio (`CONTRATO_DOMINIO.md`: convenção de saída, o que conta como movimento e custos), depois a interface comum e só então as buscas, heurísticas, gerador e experiências. A contribuição de cada membro está no Anexo B.

1.4 Estrutura do relatório

O Capítulo 2 resume a teoria de busca usada. O Capítulo 3 formula o P6 como problema de busca. O Capítulo 4 define as heurísticas, com as provas de admissibilidade e consistência. O Capítulo 5 descreve a implementação, o gerador e os testes. O Capítulo 6 apresenta o desenho experimental, os resultados e a resposta à pergunta de análise. O Capítulo 7 conclui. Os anexos contêm a declaração de uso de IA, as contribuições e excertos de código.

2 REVISÃO TEÓRICA

2.1 Problemas de busca e espaço de estados

Um problema de busca é definido por um conjunto de estados S, um estado inicial s0, uma função ACÇÕES(s) com as acções aplicáveis em s, um modelo de transição RESULTADO(s, a), um custo de acção c(s, a, s′) > 0 e um teste de objectivo (Russell & Norvig, 2021, cap. 3). Uma solução é uma sequência de acções de s0 até um estado objectivo, e é óptima se tiver o menor custo de caminho g. Os estados e as transições formam um grafo (o espaço de estados); a busca constrói uma árvore de nós, em que cada nó guarda o estado, o pai, a acção, g e a profundidade. A *busca em árvore* pode gerar o mesmo estado muitas vezes; a *busca em grafo* guarda um conjunto de estados alcançados ou fechados para o evitar, à custa de memória. As estratégias comparam-se pela completude (encontra uma solução se existir), optimalidade (encontra a de menor custo) e complexidade em tempo e espaço, expressas em função do factor de ramificação b, da profundidade d da solução mais rasa, da profundidade máxima m e do custo óptimo C*.

2.2 Estratégias de busca não informada

A **BFS** expande por níveis com uma fila FIFO; com teste de objectivo na geração encontra a solução com menos acções, que só é óptima quando todos os custos são iguais. A **DFS** expande o nó mais profundo (pilha LIFO); usa pouca memória, mas não é completa sem controlo de ciclos e não é óptima. O **IDS** repete DFS com limite 0, 1, 2, …, juntando a memória linear da DFS à completude e optimalidade (custos unitários) da BFS, à custa de re-expandir os níveis superiores. A **Custo Uniforme (UCS)** expande o nó de menor g com uma fila de prioridade e é óptima para custos positivos (Dijkstra, 1959).

Tabela 1 – Propriedades das estratégias de busca

Algoritmo	Completo?	Óptimo?	Tempo	Espaço
BFS	Sim (b finito)	Só com custos iguais	O(b^d)	O(b^d)
DFS	Não (sim em grafo finito com controlo de ciclos e limite ≥ d)	Não	O(b^m)	O(bm)
IDS	Sim	Só com custos iguais	O(b^d)	O(bd)
UCS	Sim (custos ≥ ε > 0)	Sim	O(b^(1+⌊C*/ε⌋))	O(b^(1+⌊C*/ε⌋))
Gulosa	Em grafo finito sim; em árvore não	Não	O(b^m)	O(b^m)
A*	Sim	Sim (h admissível; consistente em grafo)	Exponencial no erro de h	O(b^d)
IDA* (+TT)	Sim	Sim (h admissível)	O(b^d) por iteração; nº de iterações = valores f distintos	O(bd) (+ tabela)

2.3 Estratégias de busca informada e heurísticas

A busca informada usa uma função heurística h(n), que estima o custo de n até ao objectivo. A **Gulosa** expande o nó de menor h: é rápida mas não óptima. O **A*** expande o nó de menor f(n) = g(n) + h(n) (Hart, Nilsson & Raphael, 1968). Uma heurística é **admissível** se h(n) ≤ h*(n) para todo n, em que h* é o custo óptimo real; é **consistente** se h(n) ≤ c(n, a, n′) + h(n′) para toda a transição e h(objectivo) = 0. Consistência implica admissibilidade e garante que o A* em grafo nunca precisa de reabrir nós fechados. Se h1(n) ≤ h2(n) para todo n, h2 **domina** h1 e o A* com h2 nunca expande mais nós (salvo desempates). A qualidade mede-se pelo **factor de ramificação efectivo b***, que satisfaz N + 1 = 1 + b* + (b*)² + … + (b*)^d para N nós expandidos e solução de profundidade d. Boas heurísticas obtêm-se como custo exacto de um **problema relaxado** (com menos restrições) ou de bases de dados de padrões (Culberson & Schaeffer, 1998).

2.4 Algoritmo avançado atribuído

O **IDA*** (Korf, 1985) é o equivalente do IDS para busca informada: faz DFS em que um nó é cortado quando f = g + h excede um limite; o limite inicial é h(s0) e, em cada nova iteração, passa a ser o menor f que excedeu o limite anterior. Com h admissível, a primeira solução

encontrada é óptima, e a memória é linear na profundidade, ao contrário do A*, que guarda todos os nós gerados. O preço é re-expandir os mesmos nós em cada iteração e, sobretudo, não detectar transposições: no *Rush Hour*, o mesmo estado é alcançado por muitas ordens de movimentos diferentes, e o IDA* simples explora cada uma delas.

A **tabela de transposição** (TT) (Reinefeld & Marsland, 1994) guarda, durante uma iteração, o menor g com que cada estado foi visitado. Se um estado reaparece com $g' \geq g$ guardado, a subárvore pode ser podada: na primeira visita foi explorada com folga (limite − g) maior ou igual, logo não contém nenhuma solução dentro do limite que a segunda visita pudesse encontrar, e os valores f que excederam o limite já foram registados para o limite seguinte. A tabela é limpa no início de cada iteração (com um limite maior, uma subárvore cortada antes pode ter agora uma solução). O pseudocódigo implementado é:

```
limite ← h(s0)
repetir:
    TT ← {}                                  # limpa em cada iteração
    (sol, novo) ← DFS(raiz, limite)
    se sol: devolver sol      senão se novo = ∞: devolver falha
    limite ← novo                             # menor f > limite

DFS(n, limite):
    f ← g(n) + h(n);  se f > limite: devolver (nada, f)
    se objectivo(n): devolver (n, f)
    se TT[n.estado] ≤ g(n): devolver (nada, ∞) # transposição já explorada
    TT[n.estado] ← g(n)
    para cada filho n' fora do caminho actual, por ordem de f crescente:
        (sol, t) ← DFS(n', limite);  se sol: devolver sol;  mínimo ← min(mínimo, t)
    devolver (nada, mínimo)
```

O IDA* é indicado quando a memória é o recurso limitante, os custos tomam poucos valores distintos (cada valor distinto de f obriga a uma iteração) e a heurística é boa. Com custos variados (como o custo ponderado) o número de iterações aumenta, o que é visível nos resultados.

2.5 Abordagens alternativas e trabalhos relacionados

O *Rush Hour* clássico (6×6) foi resolvido de forma exaustiva: Collette, Raskin & Servais (2006) enumeraram todas as configurações e identificaram os puzzles mais difíceis (93 movimentos no critério de movimentos de um veículo de várias casas), usando BFS para trás a partir dos objectivos, a mesma ideia do nosso gerador. Em geral, o problema é PSPACE-completo (Flake & Baum, 2002), pelo que não se espera um algoritmo polinomial. Alternativas à busca no espaço de estados são a formulação como planeamento clássico (PDDL) resolvida por planeadores com heurísticas independentes do domínio, a satisfação de restrições/SAT com horizonte limitado e as metaheurísticas (algoritmos genéticos, *simulated annealing*), que encontram soluções mas sem garantia de optimalidade. Para tabuleiros até 8×8, a busca

heurística óptima (A*/IDA*) é a escolha adequada: os espaços alcançáveis são da ordem de 10^3 a 10^6 estados e precisamos de soluções óptimas para os dois critérios.

3 FORMULAÇÃO DO PROBLEMA DE BUSCA

3.1 Descrição do problema e pressupostos

O parque é uma grelha n×n ($2 \le n \le 8$; base 6×6) com uma saída na borda direita da linha de saída r (na base, a linha 3, índice 2 a contar de zero). Cada veículo tem identificador, orientação (H ou V), comprimento 2 ou 3 e linha (H) ou coluna (V) fixa. Os obstáculos (veículos da polícia) ocupam casas fixas. Pressupostos, registados em `CONTRATO_DOMINIO.md`:

- coordenadas a contar de zero; X é horizontal, tem 2 casas e está na linha de saída;
- **o objectivo é X tocar a borda direita** (a sua casa direita na última coluna); a saída física não é uma acção adicional, porque custaria o mesmo em todas as soluções;
- **um movimento** desloca um único veículo $k \ge 1$ casas livres na sua orientação; todas as casas atravessadas têm de estar livres (não há saltos);
- os estados objectivo são terminais (não têm acções);
- os obstáculos são imóveis e nunca estão à frente de X na linha de saída em instâncias geradas (isso tornaria o puzzle impossível);
- dois critérios: **movimentos** (cada acção custa 1) e **ponderado** (L × k, em que L é o comprimento do veículo: os camiões gastam mais).

Os dados são os do enunciado (instância base) ou gerados pelo `gerador.py` com a semente do grupo, 2026 × 100 + 6 = 202606; não foram usados dados externos.

3.2 Representação do estado

Separa-se o que muda do que é fixo. A parte fixa (tamanho, linha de saída, obstáculos e, para cada veículo, `Veiculo(id, orientacao, comprimento, fixa)` como `dataclass(frozen=True)`) fica no objecto `RushHour`. O **estado** é apenas um `tuple` de inteiros, com uma posição por veículo, pela ordem do JSON: a coluna mais à esquerda nos horizontais e a linha de cima nos verticais.

A representação é **suficiente**: como a orientação, o comprimento e a linha/coluna fixa não mudam, a posição de referência determina todas as casas ocupadas por cada veículo, e a grelha pode ser reconstruída (`grelha(s)`). É **mínima**: guardar a grelha (n^2 caracteres) ou as casas ocupadas seria redundante, e cada veículo só tem um grau de liberdade. Como é um tuplo de inteiros, é imutável e *hashable* e pode ser usado directamente em `set`/`dict` (fechados, tabela de transposição) com custo O(nº de veículos).

3.3 Estado inicial

Na instância base, a ordem dos veículos é A, B, C, O, P, Q, R, X e o estado inicial é s0 = (0, 4, 4, 0, 1, 1, 2, 1):

```
A A . . . O
P . . Q . O
P X X Q . O  → saída
P . . Q . .
B . . . C C
B . R R R .
```

3.4 Acções e modelo de transição

Há um único tipo de acção, `Accao(veiculo, deslocamento)`, com deslocamento inteiro $k \neq 0$ (positivo: direita/baixo; negativo: esquerda/cima). `accoes(s)` gera, para cada veículo e cada sentido, todos os k até à primeira casa ocupada ou à borda.

Tabela 2 – Acções do problema

Acção	Parâmetros	Pré-condições	Efeito no estado	Custo
Mover veículo	v (id), $k \in \mathbb{Z} \setminus \{0\}$	s não é objectivo; $0 \leq pos(v)+k \leq n-L(v)$; todas as $\lvert k \rvert$ casas atravessadas na linha/coluna de v estão livres (sem veículos nem obstáculos)	s' = s com $pos(v) \leftarrow pos(v) + k$; restantes posições iguais	movimentos: 1; ponderado: $L(v)\cdot\lvert k \rvert$

Na base, s0 tem 11 acções aplicáveis (A+1, A+2, A+3, C−1, C−2, C−3, O+1, Q±1, R±1); B, P e X estão presos. O factor de ramificação médio observado em todos os estados alcançáveis da base é 10,9 acções por estado. As acções são **reversíveis** (mover v por −k desfaz a acção, com o mesmo custo), o que o gerador e o cálculo de h* aproveitam.

3.5 Função de custo

$c(s, a, s') = 1$ no critério **movimentos** e $c(s, a, s') = L(v)\cdot\lvert k \rvert$ no critério **ponderado** (unidades: casas × comprimento, um "esforço" de manobra). Os custos são positivos, mas no critério ponderado variam entre 2 (carro, 1 casa) e $3\cdot(n-3)$ (camião a atravessar o tabuleiro). Consequências: a BFS minimiza o número de acções e só é óptima no primeiro critério; a DFS não é óptima em nenhum. No critério ponderado são necessários UCS, A* ou IDA*. Como o conjunto de acções é o mesmo nos dois critérios e só a função de custo muda, a comparação é justa: "movimento" significa sempre deslocar um veículo qualquer número de casas.

3.6 Teste de objectivo

Em linguagem matemática: objectivo(s) $\Leftrightarrow$ pos_X(s) + 2 = n (a casa direita de X está na coluna n − 1 da linha r). Em Python: `s[self._indices["X"]] + 2 == self.tamanho`. Há **vários**

estados objectivo (qualquer configuração dos outros veículos compatível com X na saída); na base, 4 dos estados alcançáveis são objectivos.

3.7 Dimensão do espaço de estados

Ignorando colisões, um veículo de comprimento L tem n − L + 1 posições, logo $|S| \leq \prod_i (n - L_i + 1)$. Na base (4 carros com 5 posições e 4 camiões com 4 posições) o limite é $5^4 \cdot 4^4 = 160\,000$. Por BFS exaustiva medimos **1079 estados alcançáveis** (0,7 % do limite), com 11714 transições. Num tabuleiro 8×8 com 15 veículos o limite ultrapassa 10^{12}, e as instâncias grandes geradas têm componentes de centenas de milhares de estados. Assim, BFS e UCS (que guardam todos os estados alcançados) só são viáveis enquanto os estados com distância < d ao início couberem no limite de tempo (da ordem de 10^5 estados em 60 s, em Python), e DFS/IDS (busca em árvore, O(b^d) com b ≈ 5 a 11) só com soluções curtas ($d \lesssim 8$). As heurísticas reduzem o número de estados expandidos (Capítulo 6).

4 HEURÍSTICAS

4.1 Definição das heurísticas

Seja r a linha de saída, x a coluna esquerda de X e d_X = n − 2 − x a distância que X ainda tem de percorrer. B(s) é o conjunto de veículos distintos que ocupam casas (r, c) com c ≥ x + 2 (à frente de X). Para um bloqueador vertical b de comprimento L_b, m_b é o menor deslocamento que o tira da linha r para cima ou para baixo, ignorando os outros veículos mas respeitando as bordas e os obstáculos da sua coluna. Se um obstáculo ou um veículo horizontal estiver à frente de X, ou um bloqueador não puder sair da linha, o estado não tem solução e todas as heurísticas valem ∞.

- **h1(s) = |B(s)|**, o nº de veículos que bloqueiam X (dica do enunciado). Relaxação: cada bloqueador sai com uma acção. Na base, h1(s0) = 2 (Q e O).
- **h2(s) = h1(s) + $|B_2(s)|$**, em que B_2 são os veículos distintos, fora de B, que ocupam as casas que cada bloqueador atravessa ao sair pelo lado mais curto (dica do enunciado). h2(s0) = 4.
- **h3(s) = h1(s) + [d_X > 0]**: acrescenta o movimento do próprio X, que é uma acção distinta das dos bloqueadores. h3(s0) = 3.
- **h4(s)**, a relaxação “cada veículo sozinho no tabuleiro”: o custo óptimo do problema em que os veículos não se bloqueiam uns aos outros. No critério ponderado, h4(s) = 2·d_X + Σ_(b∈B) L_b·m_b; em movimentos, h4 = h3. Na base, h4(s0) = 2·3 + 3·2 + 3·3 = 21 no ponderado (óptimo 40).

h1 e h2 são as dicas do enunciado; h3 e h4 são as heurísticas propostas pelo grupo, que as melhoram: h3 domina h1 em movimentos, h4 domina h1 e h3 no ponderado, e ambas são consistentes nos dois critérios, ao contrário de h2.

4.2 Prova de admissibilidade

Lema. Em qualquer solução a partir de s, cada b ∈ B(s) é movido pelo menos uma vez e o seu deslocamento total é ≥ m_b; X percorre pelo menos d_X casas. *Prova:* X só se move na linha r e não salta veículos, logo para chegar a x = n − 2 tem de ocupar sucessivamente todas as casas (r, x+2), …, (r, n−1). No momento em que X entra na casa ocupada inicialmente por b, b já não está na linha r; como b é vertical, isso exige deslocá-lo pelo menos m_b casas (os obstáculos não se movem, por isso um lado bloqueado por um obstáculo nunca fica livre). □

h1. Cada acção move um só veículo e custa ≥ 1, logo uma solução tem ≥ |B(s)| acções e h1 ≤ h* (no ponderado cada acção custa ≥ 2, folga ainda maior). **h3.** Se d_X > 0, X move-se pelo menos uma vez, e essa acção é distinta das acções dos |B| bloqueadores: h3 ≤ h*. **h4.** No ponderado, o custo de uma solução é a soma, por veículo, de L·Σ|k|. Pelo lema, a parcela de X é ≥ 2·d_X e a de cada b ∈ B é ≥ L_b·m_b; as parcelas são de veículos diferentes, logo somam-se: h4 ≤ h*. Os estados com h = ∞ não têm solução (h* = ∞).

h2 não é admissível no critério movimentos. Contra-exemplo (5×5, linha de saída 3), verificado em `tests/contra_exemplo_h2.py`:

```
. . . C C
. . . . A
X X B . A   → saída
. . B . .
. E E . .
```

B = {B, A}, logo h1 = 2. Pelo lado mais curto, B desce 1 casa (E bloqueia) e A sobe 1 casa (C bloqueia), logo h2 = 4. Mas B−2, A+2, X+3 resolve em 3 movimentos: h* = 3 < h2 = 4. O erro da dica é assumir que o bloqueador sai pelo lado mais curto, quando o lado mais longo pode estar livre. **h2 é admissível no critério ponderado:** o caminho mais curto de b tem m_b casas, logo no máximo m_b bloqueadores de 2.º nível; como L_b ≥ 2 e m_b ≥ 1, 1 + m_b ≤ L_b·m_b; somando sobre b ∈ B, h2 ≤ Σ L_b·m_b ≤ h4 ≤ h*. Isto responde à exigência específica 2 do P6.

4.3 Prova de consistência

Uma acção move um só veículo v por k casas. (i) Se v = X, as casas para onde X avança estavam livres e as que liberta não ficam à sua frente, logo B não muda; d_X varia exactamente k: h1 não muda, [d_X > 0] diminui no máximo 1 ≤ c, e 2·d_X diminui no máximo 2|k| = c. (ii)

Se v ≠ X, só a parcela de v muda: h1 e h3 diminuem no máximo 1 ≤ c (v sai de B); em h4, m_v é a distância a uma posição livre da linha r, que varia no máximo |k| quando v se desloca |k| casas, logo a parcela L_v·m_v diminui no máximo L_v|k| = c (se v entra em B, h aumenta, o que não viola a condição). Em todos os casos h(s) ≤ c(s, a, s′) + h(s′), e h = 0 nos objectivos (B = ∅, d_X = 0): **h1, h3 e h4 são consistentes nos dois critérios**. **h2 não é consistente em movimentos** (inadmissível implica inconsistente); por isso o A* implementado reabre nós fechados quando encontra um g melhor, mantendo a correcção com qualquer heurística.

4.4 Verificação empírica e dominância

O teste automático (`heuristicas.verificar`, chamado nos testes `pytest` e em `experiencias.py`) enumera todos os estados alcançáveis, calcula h* exacto por Dijkstra no grafo invertido a partir de todos os objectivos e verifica h(s) ≤ h*(s) em todos os estados e h(s) ≤ c + h(s′) em todas as transições. Na base e nas 10 instâncias pequenas foram verificados, em cada critério, 6 321 estados e 47 306 transições: h1, h3 e h4 não violam a admissibilidade nem a consistência; h2 tem 0 violações de admissibilidade e 262 de consistência em movimentos, e 0 de admissibilidade e 0 de consistência no ponderado. A inadmissibilidade de h2 é rara (não aparece nestas instâncias, só no contra-exemplo construído), mas a inconsistência é frequente. A dominância h1 ≤ h3 ≤ h4 verifica-se em todos os estados (teste `test_dominancia`). A Tabela 3 compara as heurísticas no A* (nós expandidos e b*) em instâncias representativas.

Tabela 3 – Comparação das heurísticas no A*

Instância	d	Exp. h0	Exp. h1	Exp. h2	Exp. h3	Exp. h4	b* h1	b* h4
base (mov.)	8	1057	783	357	530	530	2,12	2,01
pequena_01 (mov.)	7	334	184	112	114	114	1,90	1,75
pequena_02 (mov.)	7	46	31	29	21	21	1,38	1,28
pequena_05 (mov.)	8	147	118	90	81	81	1,61	1,52
media_01 (mov.)	15	3849	2977	2666	2673	2673	1,60	1,58
media_04 (mov.)	20	8013	7278	7097	7167	7167	1,47	1,47
media_06 (mov.)	22	8442	4614	4071	4283	4283	1,38	1,38
grande_01 (mov.)	17	81433	36168	23242	26511	26511	1,76	1,73
grande_02 (mov.)	15	14291	12713	10851	12045	12045	1,78	1,77
base (pon.)	11	1071	1069	1069	1065	816	2,22	1,70
pequena_01 (pon.)	7	409	368	346	341	200	2,12	1,92
pequena_02 (pon.)	8	56	48	48	45	28	1,40	1,28
pequena_05 (pon.)	8	144	136	135	131	99	1,65	1,57
media_01 (pon.)	16	4002	3873	3754	3773	2957	1,57	1,54
media_04 (pon.)	26	8125	8051	8045	7969	7354	1,40	1,34
media_06 (pon.)	27	6651	5932	5670	5627	4099	1,34	1,29

grande_01 (pon.)	21	94021	80794	74068	74755	32405	1,68	1,56
grande_02 (pon.)	19	16697	15664	15321	15078	12279	1,73	1,55

d é o nº de movimentos da solução encontrada pelo A*; b* calcula-se com N = nós expandidos. "—" indica que a execução excedeu 60 s. No critério movimentos, h2 expande menos nós do que h3/h4 porque é maior e por vezes sobrestima; na `grande_05` isso fez o A* com h2 devolver 31 movimentos quando o óptimo é 30. h3 e h4 coincidem em movimentos. No critério ponderado, h4 é a mais informada e a única que reduz claramente o b*.

5 IMPLEMENTAÇÃO

5.1 Arquitectura do código

O código segue a estrutura mínima do enunciado (Figura 1). `problema.py` define a classe abstracta `Problema` (estado_inicial, accoes, resultado, custo, e_objectivo, heuristica), o nó `No(estado, pai, accao, g, profundidade)` e `caminho(no)`. `dominio.py` contém `RushHour`, subclasse de `Problema`. `buscas.py` contém os algoritmos, que **só usam a interface**: trocar de problema não obriga a alterar nenhum algoritmo (os testes usam também tabuleiros 4×4 e 5×5 sem mudar `buscas.py`). `heuristicas.py` define h0–h4 e as ferramentas de análise (h* exacto, verificação, b*, divergência entre critérios). `gerador.py`, `experiencias.py` e `main.py` usam estes módulos.

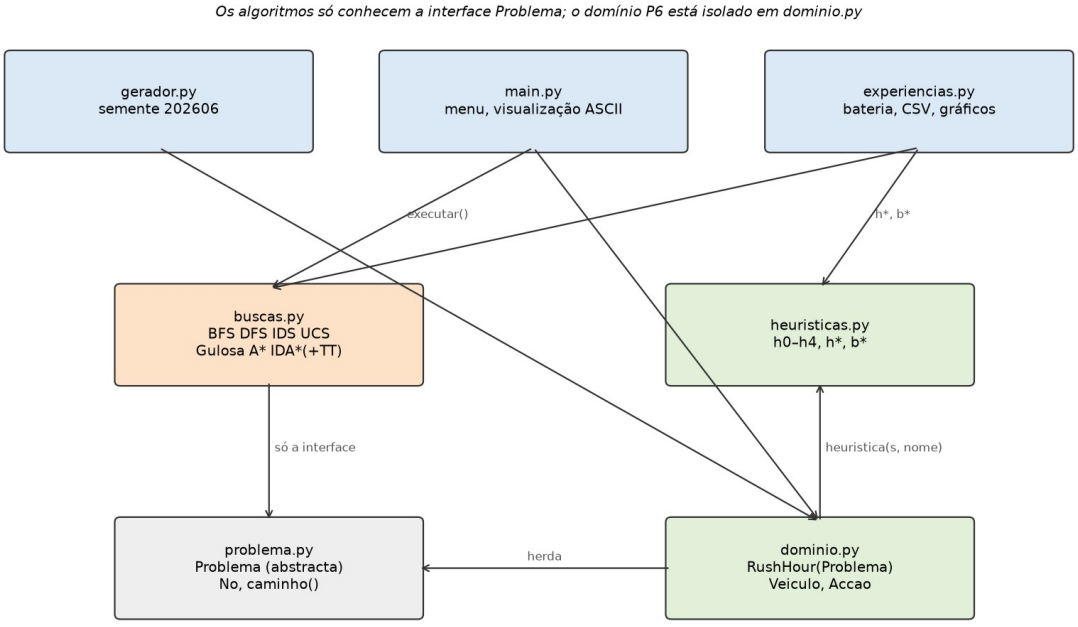


Figura 1 – Arquitectura do programa

5.2 Estruturas de dados

- **Fronteira:** `collections.deque` na BFS (FIFO, O(1) nas duas pontas); `heapq` no UCS, Gulosa e A*, com entradas (prioridade, contador, nó): o contador desempata em ordem de inserção e evita comparar nós; recursão (pilha implícita) na DFS, IDS e IDA*.
- **Alcançados/fechados:** `set` de estados (tuplos) para teste de pertença em O(1) médio; `dict` `melhor_g` estado → menor g na fila de prioridade.
- **Entradas obsoletas:** em vez de actualizar a prioridade dentro do *heap* (O(n)), insere-se uma nova entrada quando se encontra um g melhor e, ao retirar, descarta-se a entrada cujo g é maior do que `melhor_g[estado]` (descarte preguiçoso, O(log n) por operação).
- **Controlo de ciclos no caminho** (DFS, IDS, IDA*): `set` com os estados do caminho actual, actualizado ao entrar e sair da recursão: O(1) por teste e memória O(d).
- **Tabela de transposição:** `dict` estado → menor g na iteração actual, limpo em cada iteração.
- **Reconstrução do caminho:** cada nó guarda o pai; `caminho(no)` segue os pais até à raiz (O(d)), sem copiar caminhos em cada nó.
- **Domínio:** `resultado()` só verifica as casas atravessadas, com uma *cache* da ocupação do último estado (os algoritmos aplicam várias acções seguidas ao mesmo estado). Medimos uma redução de cerca de 5× no tempo por nó face à versão inicial, que regenerava todas as acções para validar cada uma.

5.3 Algoritmos implementados

Todos os algoritmos devolvem um `Resultado` com o nó final, o estado da execução (ok, sem solução, limite, tempo) e as mesmas métricas: nós expandidos (nós cujos sucessores foram gerados), gerados (sucessores criados, mais a raiz), fronteira máxima e re-expandidos. O prazo de 60 s é verificado dentro do ciclo principal (de 256 em 256 expansões). Decisões:

- **BFS:** busca em grafo com teste de objectivo **na geração**, o que poupa um nível inteiro de expansões; o `set` de alcançados inclui a fronteira.
- **DFS:** recursiva, com controlo de ciclos no caminho actual e limite de profundidade configurável (por omissão 50); devolve “limite” se cortou ramos e “sem solução” se esgotou a árvore sem cortes.
- **IDS:** DFS limitada com limite 0, 1, 2, …; conta como re-expansão cada expansão de um estado já expandido numa iteração anterior; pára com “sem solução” quando uma iteração termina sem cortes.

- **UCS / Gulosa / A*:** um único `_melhor_primeiro` parametrizado pela prioridade (g, h ou g + h) e com teste de objectivo **na expansão** (necessário para a optimalidade). O A* desempata f igual pelo maior g (mais perto do objectivo) e reabre nós fechados se encontrar g menor, o que só acontece com h2.
- **IDA* (+TT):** recursivo, filhos ordenados por f crescente, controlo de ciclos no caminho e, na variante com tabela, poda por transposição; `idastar_simples` é a mesma função sem tabela, para medir a redução de nós (exigência específica 3).

Excerto do A* (o código completo está no repositório e no Anexo C):

```
while fronteira:
    _, _, no = heapq.heappop(fronteira)
    if no.g > melhor_g[no.estado]:
        continue                                # entrada obsoleta
    if no.estado in fechados:
        if not reabrir: continue
        r.reexpandidos += 1                     # só com h inconsistente
    if problema.e_objectivo(no.estado):         # teste na expansão
        return no
    fechados.add(no.estado); r.expandidos += 1
    for a in problema.accoes(no.estado):
        filho = _filho(problema, no, a); r.gerados += 1
        antigo = melhor_g.get(filho.estado)
        if antigo is not None and filho.g >= antigo: continue
        melhor_g[filho.estado] = filho.g
        heapq.heappush(fronteira, (prioridade(filho), next(desempate), filho))
```

5.4 Gerador de instâncias

O `gerador.py` cria puzzles por **busca para trás a partir de estados objectivo**: (1) sorteia, com `random.Random("202606-<nível>")`, um tamanho, carros, camiões e obstáculos numa configuração em que X já está no objectivo; (2) enumera por BFS toda a componente de estados alcançáveis a partir dela (as acções são reversíveis, logo são exactamente os estados que chegam a esse objectivo); (3) calcula, por BFS multi-origem a partir de **todos** os objectivos da componente, a distância óptima em movimentos de cada estado; (4) escolhe como estado inicial um dos estados mais distantes. A distância é exacta (BFS), por isso o gerador **certifica** o óptimo e não apenas o número de passos do passeio para trás. Instâncias repetidas são rejeitadas, e cada JSON guarda a semente, o nível, o óptimo certificado e o tamanho da componente.

Níveis: **média** e **grande** só aceitam puzzles com óptimo ≥ 15 movimentos (regra 2 do P6), e distinguem-se pelo tamanho da componente (3 000–40 000 estados nas médias, em tabuleiros 6×6/7×7; 250 000–800 000 nas grandes, em 7×7/8×8). Na **pequena** o enunciado exige que todos os algoritmos terminem. Verificámos que isso é incompatível com o mínimo de 15 movimentos: DFS e IDS fazem busca em árvore com controlo de ciclos só no caminho, e com b ≈ 5–11 e d = 15 a árvore tem 10^{10}–10^{15} nós (em puzzles com apenas 553 estados alcançáveis e d = 15, o IDS excedeu os 60 s com mais de um milhão de expansões). Por isso, a pequena

aceita o estado mais distante (óptimo ≥ 6) que DFS e IDS ainda resolvem dentro de um orçamento **determinístico** de 300 000 nós; a decisão fica documentada no código e não relaxa a regra dos 15 movimentos nos outros níveis. Também suporta M×M até 8×8 e obstáculos (exigência 1). Comando: `python gerador.py --nivel todos`.

5.5 Testes automáticos

A pasta `tests/` contém testes `pytest` em quatro ficheiros. Resultado da execução final: **68 passed, 2 skipped**.

- `test_dominio.py` (Lino): instância base, movimentos e imutabilidade, movimentos ilegais, saída, obstáculos, colisões, tabuleiro 8×8, `caminho()`;
- `test_buscas.py`: **sucessores de um estado conhecido** (as 11 acções de s0); óptimos da base (8 e 40) com UCS, A* h0/h1/h3/h4 e IDA*; **A* e UCS devolvem o mesmo custo** em instâncias geradas; BFS óptima em movimentos; **instância sem solução** em todos os algoritmos; limite da DFS; re-expansões do IDS; a TT reduz nós; Gulosa válida; limite de tempo; divergência dos critérios; cada solução é revalidada reaplicando as acções;
- `test_heuristicas.py`: **admissibilidade e consistência em todos os estados** da base (1079 estados, dois critérios); h = 0 nos objectivos; dominância; valores conhecidos de h1/h4; estados impossíveis com h = ∞; o contra-exemplo de h2; h2 ≤ h4 no ponderado; b*;
- `test_gerador.py`: reprodutibilidade com a semente, óptimo certificado igual ao da BFS após gravar e reler o JSON e o mínimo de movimentos de todas as instâncias gravadas.

5.6 Interface e instruções de execução

Instalação: `python -m pip install -r requirements.txt` (Python ≥ 3.10). O `main.py` tem um menu de texto (instância base, ficheiro JSON ou gerada com semente e nível; critério; algoritmo; heurística; limite de tempo e de profundidade) e mostra a solução passo a passo em ASCII e as métricas. Para demonstração aceita também argumentos:

```
python main.py                                    # menu interactivo
python main.py --alg astar --h h4 --criterio ponderado
python main.py --instancia instancias/media_04.json --alg idastar --h h4 --sem-passos
python gerador.py --nivel todos                   # regenera as 30 instâncias
python experiencias.py --workers 2                # bateria completa, CSV e gráficos
python -m pytest -q                               # testes
```

Excerto da execução `python main.py --alg astar --h h4 --criterio ponderado` (últimos passos):

```
P X X . . .  → saída
B . . Q . O
B C C Q . O
R R R Q . O
```

```
Passo 11/11: mover X 3 casa(s) para direita → (custo 6, acumulado 40)
P A A . . .
P . . . . .
P . . . X X  → saída
B . . Q . O
B C C Q . O
R R R Q . O

O chapa X chegou à saída!

Solução: O+1 C-1 O+2 C-2 Q+1 A+1 P-1 B-1 R-2 Q+1 X+3

Métricas:
  estado                    ok
  custo da solução          40
  comprimento (movimentos)  11
  nós expandidos            816
  nós gerados               9111
  fronteira máxima          440
  nós re-expandidos         0
  iterações                 —
  tempo (s)                 0.1459
  memória de pico (MB)      0.489
```

6 EXPERIÊNCIAS E RESULTADOS

6.1 Desenho experimental

Máquina: 11th Gen Intel(R) Core(TM) i5-1135G7 @ 2.40GHz, 8 CPUs lógicos, 14,8 GB de RAM, Linux-7.0.0-43-generic-x86_64-with-glibc2.43, Python 3.14.4. Semente 202606. Instâncias: a base mais 10 pequenas, 10 médias e 10 grandes, geradas pelo `gerador.py` (Secção 5.4). Para cada instância e cada critério (movimentos e ponderado) correram 13 configurações: BFS, DFS (limite 50), IDS, UCS, Gulosa (h1, h2), A* (h0, h1, h2, h3, h4), IDA* simples (h4) e IDA*+TT (h4), num total de 806 execuções. Cada execução corre num processo novo, com limite de memória de 2 GB (`RLIMIT_AS`) e de 60 s (prazo verificado dentro do algoritmo), com 2 execuções em paralelo nas pequenas e médias e 3 nas grandes; as que excedem os limites aparecem como “falhou (tempo)” ou “falhou (memória)”. Mede-se o tempo com `time.perf_counter` e a memória de pico com `tracemalloc` (o tempo inclui a sobrecarga do tracemalloc, igual para todos). As contagens de nós são determinísticas, por isso basta uma repetição; o tempo de uma execução isolada tem variação de alguns %, e o desvio-padrão entre instâncias está nos CSV (`outputs/resumo_*.csv`). O óptimo de referência de cada instância é o menor custo obtido por UCS, A* ou IDA* (todos óptimos com h admissível) e coincide com o óptimo certificado pelo gerador.

6.2 Resultados

As Tabelas 4 a 9 resumem os resultados por nível e critério. As médias de custo, nós, tempo e memória usam só as execuções resolvidas; “Resolvidas” indica quantas de 10 terminaram, e o

asterisco assinala falhas (motivo nos CSV). As Figuras 2 a 6 mostram a evolução com a dimensão da instância, medida pela profundidade óptima d (nº de movimentos da solução óptima).

Tabela 4 – Resultados médios por algoritmo — instâncias pequenas, critério movimentos

Algoritmo	Heur.	Resolvidas	Custo médio	Custo / óptimo	Nós exp.	Nós ger.	Fronteira máx.	Tempo (s)	Memória (MB)
BFS	—	10/10	7,0	1,000	134	1 034	66	0,016	0,08
DFS	—	10/10	34,6	5,036	1 069	9 762	36	0,216	0,05
IDS	—	10/10	7,0	1,000	11 042	80 923	8	1,580	0,05
UCS	—	10/10	7,0	1,000	195	1 545	73	0,025	0,10
Gulosa	h1	10/10	7,4	1,052	39	287	60	0,007	0,05
Gulosa	h2	10/10	7,3	1,039	27	189	53	0,006	0,05
A*	h0	10/10	7,0	1,000	195	1 545	73	0,026	0,12
A*	h1	10/10	7,0	1,000	89	678	90	0,015	0,10
A*	h2	10/10	7,0	1,000	52	363	66	0,011	0,07
A*	h3	10/10	7,0	1,000	54	388	57	0,009	0,06
A*	h4	10/10	7,0	1,000	54	388	57	0,011	0,06
IDA* simples	h4	10/10	7,0	1,000	707	4 848	8	0,228	0,04
IDA*+TT	h4	10/10	7,0	1,000	141	956	8	0,046	0,04

Tabela 5 – Resultados médios por algoritmo — instâncias pequenas, critério ponderado

Algoritmo	Heur.	Resolvidas	Custo médio	Custo / óptimo	Nós exp.	Nós ger.	Fronteira máx.	Tempo (s)	Memória (MB)
BFS	—	10/10	21,6	1,024	134	1 034	66	0,016	0,08
DFS	—	10/10	74,7	3,607	1 069	9 762	36	0,208	0,05
IDS	—	10/10	21,6	1,024	11 042	80 923	8	1,560	0,05
UCS	—	10/10	21,1	1,000	239	1 908	73	0,031	0,11
Gulosa	h1	10/10	21,9	1,031	39	287	60	0,007	0,05
Gulosa	h2	10/10	21,9	1,031	27	189	53	0,006	0,05
A*	h0	10/10	21,1	1,000	239	1 908	73	0,034	0,13
A*	h1	10/10	21,1	1,000	221	1 766	80	0,034	0,13
A*	h2	10/10	21,1	1,000	212	1 694	84	0,039	0,13
A*	h3	10/10	21,1	1,000	202	1 626	79	0,033	0,13
A*	h4	10/10	21,1	1,000	78	582	66	0,015	0,08
IDA* simples	h4	10/10	21,1	1,000	17 110	126 335	11	6,430	0,05
IDA*+TT	h4	10/10	21,1	1,000	397	2 780	11	0,157	0,05

Tabela 6 – Resultados médios por algoritmo — instâncias médias, critério movimentos

Algoritmo	Heur.	Resolvidas	Custo médio	Custo / óptimo	Nós exp.	Nós ger.	Fronteira máx.	Tempo (s)	Memória (MB)
BFS	—	10/10	19,4	1,000	7 199	90 319	1 566	1,400	2,05
DFS	—	0/10 *	—	—	—	—	—	—	—
IDS	—	0/10 *	—	—	—	—	—	—	—
UCS	—	10/10	19,4	1,000	8 035	100 431	1 774	1,872	2,75
Gulosa	h1	10/10	21,9	1,127	4 356	56 198	1 952	1,030	1,94
Gulosa	h2	10/10	21,3	1,089	3 996	52 716	1 920	1,163	1,90
A*	h0	10/10	19,4	1,000	8 035	100 431	1 774	1,764	3,24
A*	h1	10/10	19,4	1,000	5 959	74 390	2 147	1,559	2,91
A*	h2	10/10	19,4	1,000	5 363	67 423	2 051	1,497	2,67
A*	h3	10/10	19,4	1,000	5 513	68 952	1 811	1,175	2,53
A*	h4	10/10	19,4	1,000	5 513	68 952	1 811	1,380	2,53
IDA* simples	h4	0/10 *	—	—	—	—	—	—	—
IDA*+TT	h4	5/10 *	16,0	1,000	18 392	171 005	17	12,960	0,50

Tabela 7 – Resultados médios por algoritmo — instâncias médias, critério ponderado

Algoritmo	Heur.	Resolvidas	Custo médio	Custo / óptimo	Nós exp.	Nós ger.	Fronteira máx.	Tempo (s)	Memória (MB)
BFS	—	10/10	67,9	1,002	7 199	90 319	1 566	1,447	2,05
DFS	—	0/10 *	—	—	—	—	—	—	—
IDS	—	0/10 *	—	—	—	—	—	—	—
UCS	—	10/10	67,8	1,000	8 030	100 127	1 794	1,740	2,66
Gulosa	h1	10/10	74,4	1,096	4 346	56 086	1 951	1,038	1,96

Gulosa	h2	10/10	70,8	1,028	3 990	52 662	1 893	1,081	1,87
A*	h0	10/10	67,8	1,000	8 030	100 127	1 794	1,601	3,12
A*	h1	10/10	67,8	1,000	7 711	96 544	1 774	1,735	3,00
A*	h2	10/10	67,8	1,000	7 579	95 137	1 796	2,036	2,96
A*	h3	10/10	67,8	1,000	7 535	94 532	1 727	1,810	2,94
A*	h4	10/10	67,8	1,000	6 088	77 329	1 848	1,768	2,74
IDA* simples	h4	0/10 *	—	—	—	—	—	—	—
IDA*+TT	h4	4/10 *	51,8	1,000	23 037	173 190	24	16,860	0,39

Tabela 8 – Resultados médios por algoritmo — instâncias grandes, critério movimentos

Algoritmo	Heur.	Resolvidas	Custo médio	Custo / óptimo	Nós exp.	Nós ger.	Fronteira máx.	Tempo (s)	Memória (MB)
BFS	—	10/10	25,8	1,000	86 087	1 198 738	14 642	15,578	27,32
DFS	—	0/10 *	—	—	—	—	—	—	—
IDS	—	0/10 *	—	—	—	—	—	—	—
UCS	—	10/10	25,8	1,000	97 609	1 372 993	18 030	19,624	39,10
Gulosa	h1	10/10	31,5	1,209	31 249	393 730	15 948	7,258	18,62
Gulosa	h2	10/10	32,5	1,265	29 563	379 436	17 698	8,178	19,57
A*	h0	10/10	25,8	1,000	97 609	1 372 993	18 030	21,289	45,95
A*	h1	10/10	25,8	1,000	74 392	1 026 999	19 884	18,360	34,70
A*	h2	10/10	25,9	1,003	65 602	910 653	20 042	18,302	32,55
A*	h3	10/10	25,8	1,000	69 307	955 773	18 405	16,672	31,93
A*	h4	10/10	25,8	1,000	69 307	955 773	18 405	18,290	31,93
IDA* simples	h4	0/10 *	—	—	—	—	—	—	—
IDA*+TT	h4	0/10 *	—	—	—	—	—	—	—

Tabela 9 – Resultados médios por algoritmo — instâncias grandes, critério ponderado

Algoritmo	Heur.	Resolvidas	Custo médio	Custo / óptimo	Nós exp.	Nós ger.	Fronteira máx.	Tempo (s)	Memória (MB)
BFS	—	10/10	97,8	1,037	86 087	1 198 738	14 642	15,971	27,32
DFS	—	0/10 *	—	—	—	—	—	—	—
IDS	—	0/10 *	—	—	—	—	—	—	—
UCS	—	10/10	94,8	1,000	114 688	1 632 606	25 721	23,977	46,39
Gulosa	h1	10/10	108,8	1,137	31 386	395 333	15 866	6,994	18,54
Gulosa	h2	10/10	106,3	1,124	29 592	379 791	17 419	7,773	19,33
A*	h0	10/10	94,8	1,000	114 688	1 632 606	25 721	25,137	54,25
A*	h1	10/10	94,8	1,000	103 511	1 459 565	24 086	25,614	49,54
A*	h2	10/10	94,8	1,000	98 105	1 379 582	23 549	27,765	46,97
A*	h3	10/10	94,8	1,000	99 493	1 399 224	23 368	24,700	47,24
A*	h4	10/10	94,8	1,000	71 054	976 185	18 311	19,464	32,65
IDA* simples	h4	0/10 *	—	—	—	—	—	—	—
IDA*+TT	h4	0/10 *	—	—	—	—	—	—	—

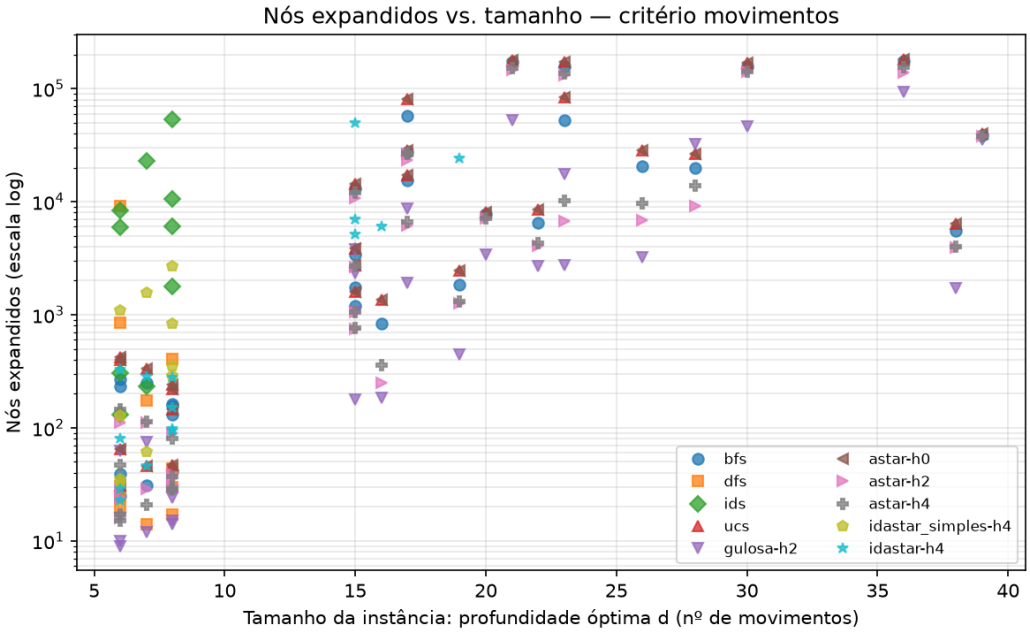


Figura 2 – Nós expandidos em função do tamanho da instância (escala logarítmica), critério movimentos

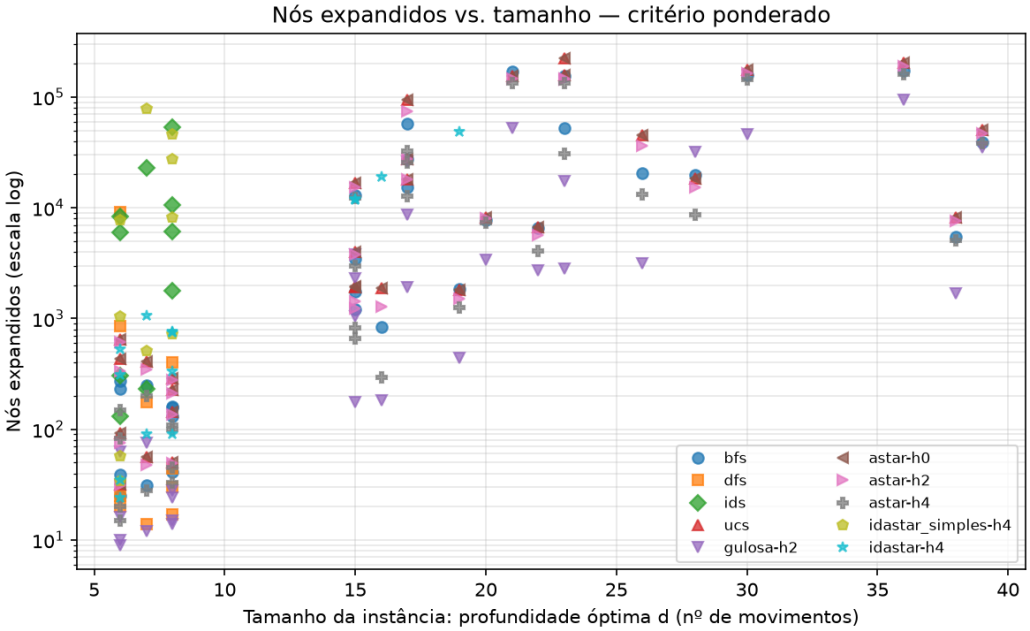


Figura 3 – Nós expandidos em função do tamanho da instância (escala logarítmica), critério ponderado

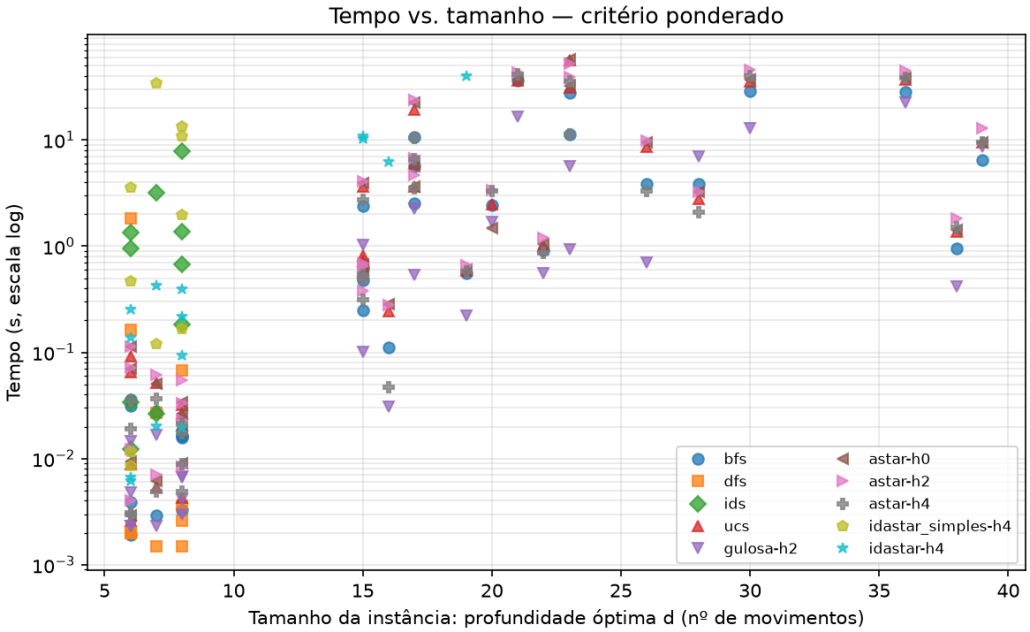


Figura 4 – Tempo de execução em função do tamanho da instância (escala logarítmica), critério ponderado

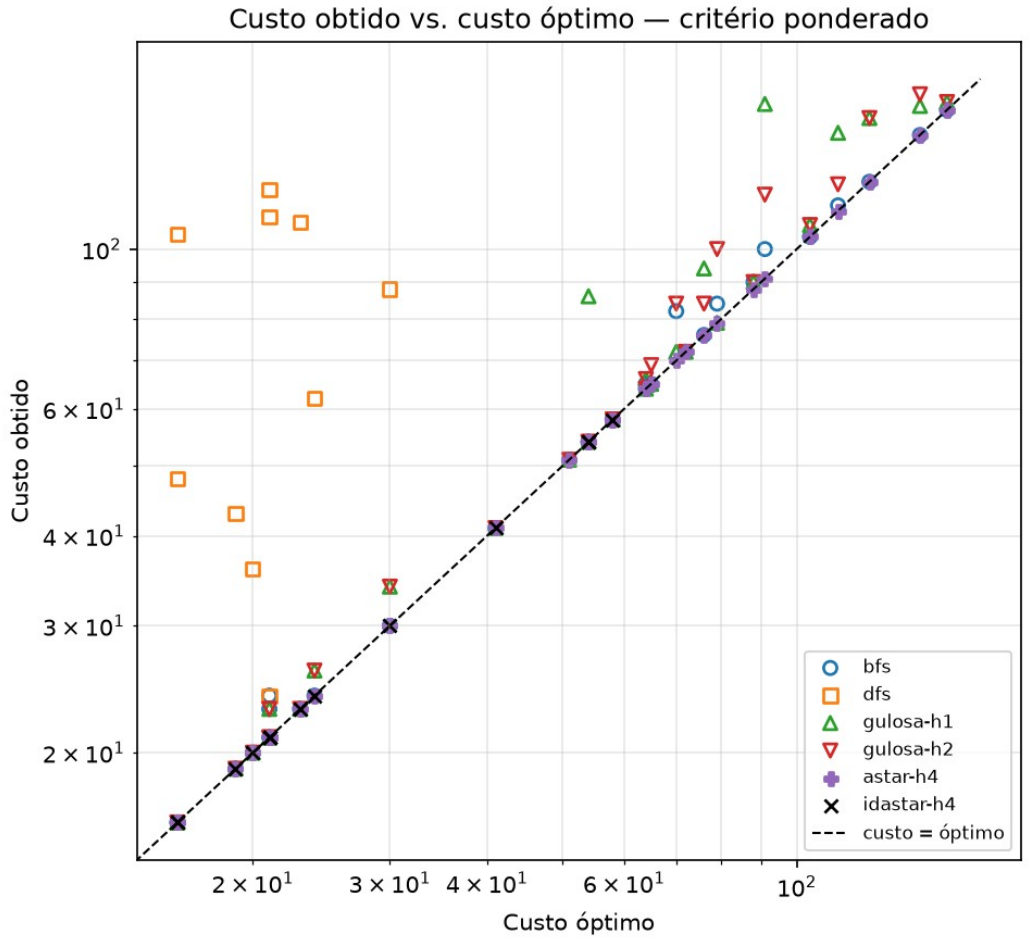


Figura 5 – Custo obtido em função do custo óptimo, critério ponderado

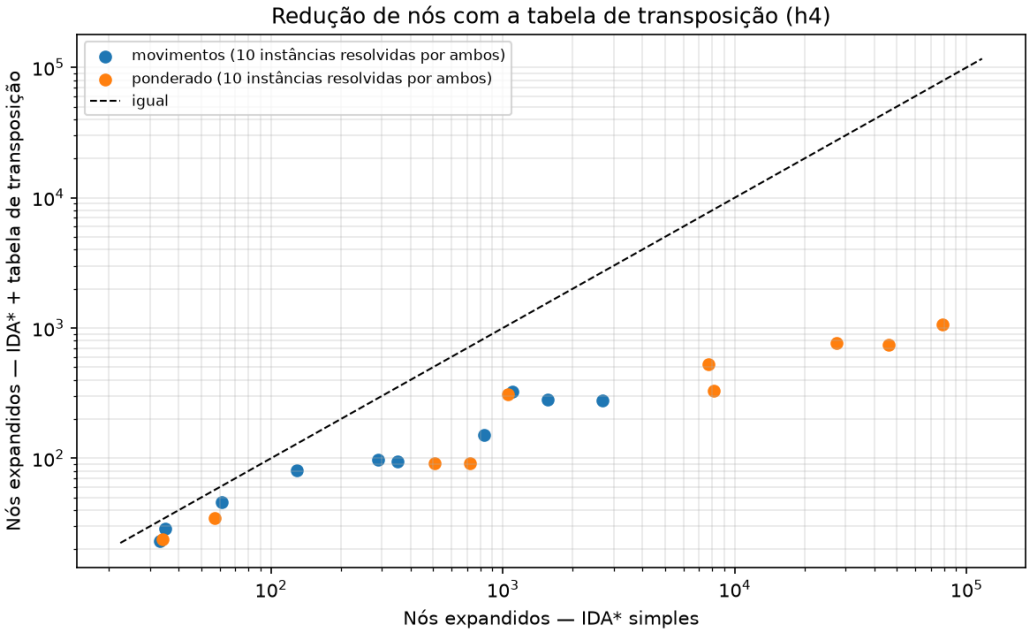


Figura 6 – Nós expandidos pelo IDA* simples e pelo IDA* com tabela de transposição

6.3 Análise e discussão

Completude e limites das buscas não informadas. Nas instâncias pequenas todos os algoritmos terminam, como o enunciado exige. Nas médias (óptimo de 15 a 38 movimentos), DFS e IDS falham por tempo em todas (DFS 0/10, IDS 0/10), embora as componentes tenham só 4 000–33 000 estados. A teoria explica-o: são buscas em árvore com controlo de ciclos apenas no caminho, e no *Rush Hour* o mesmo estado é alcançado por um número exponencial de ordens de movimentos (transposições), pelo que exploram O(b^d) nós e não O(|S|). Nas pequenas o IDS já expande em média 11 042 nós, contra 134 da BFS, e na base (d = 8, b ≈ 10,9) excede 60 s. A BFS e o UCS, sendo buscas em grafo, resolvem todas as médias em poucos segundos, porque o número de estados distintos é pequeno.

Nas instâncias grandes (componentes de 290 000 a 780 000 estados, óptimo de 15 a 36 movimentos), a BFS resolveu 10/10 e o UCS 10/10 no ponderado, contra 10/10 do A* com h4 e 10/10 da Gulosa (h2), enquanto DFS, IDS, IDA* simples e IDA*+TT não resolveram nenhuma. **Ao contrário do previsto no enunciado (“só as informadas terminam”), a BFS e o UCS ainda terminaram em todas as grandes**, com médias de 15,6 s e 24,0 s (máximo de 56 s no UCS). A razão é que só expandem os estados mais próximos do início do que a solução (em média 86 087 na BFS), uma fracção pequena da componente. Para os fazer falhar seriam precisas componentes de vários milhões de estados, que o nosso gerador não consegue certificar por enumeração (Secção 7.2). Mesmo assim, a vantagem das informadas mantém-se:

no ponderado, o A* com h4 expande 60,6 % dos nós do UCS e usa menos memória, e a Gulosa resolve em cerca de um terço do tempo, embora com custos até 27 % acima do óptimo.

Optimalidade. UCS, A* (h0, h1, h3, h4) e IDA*+TT devolveram sempre o custo óptimo (Custo/óptimo = 1,000), confirmando a teoria (h1, h3 e h4 são consistentes). O A* com h2 foi óptimo em todas as instâncias menos uma: na `grande_05` (critério movimentos) devolveu uma solução de 31 movimentos, quando o óptimo é 30. É a confirmação experimental de que h2 não é admissível em movimentos (Secção 4.2): a sobrestimação fez o A* fechar o objectivo antes de expandir o caminho óptimo. No ponderado, onde provámos $h2 \leq h4 \leq h^*$, o A* com h2 foi sempre óptimo. A BFS é óptima em movimentos, mas no critério ponderado o custo médio foi 1,024 vezes o óptimo nas pequenas: minimiza acções e não custo. A DFS devolve soluções muito piores (até 50 movimentos para óptimos de 6 a 8), porque segue o primeiro ramo até ao limite de 50. A Gulosa resolve depressa com poucos nós, mas sem óptimo (até ≈ 10 % acima nas médias, Figura 5).

Heurísticas. No critério movimentos, o A* com h1 expande em média 36,4 % menos nós do que com h0 (UCS) nas médias, e h2/h3/h4 melhoram um pouco mais (Tabela 3). Os ganhos são modestos porque o *Rush Hour* tem poucos estados por nível de profundidade: muitos estados têm o mesmo f, e o que domina é o número de estados com $f < C^*$. No critério ponderado, h1–h3 quase não ajudam (A* h3 expande 85,2 % dos nós do UCS), porque contam acções (≈ 1 por veículo) enquanto o custo de cada acção vale 2 a 15. A h4 mede o custo mínimo em casas × comprimento e expande 61,3 % dos nós do UCS nas médias (10 instâncias), e ainda menos nas pequenas e grandes. Isto confirma a regra de que uma heurística deve estar na mesma escala da função de custo.

IDA* e tabela de transposição. A tabela de transposição reduziu os nós expandidos do IDA* para 42,8 % (movimentos) e 20,9 % (ponderado) dos do IDA* simples nas pequenas (Figura 6, exigência específica 3), e o IDA* simples não resolveu nenhuma média, enquanto o IDA*+TT resolveu 5/10 em movimentos. Mesmo assim, o IDA* expande mais nós do que o A*: repete as iterações (uma por valor distinto de f) e a tabela é limpa em cada uma. No critério ponderado o número de iterações aumenta (f toma muitos valores), e por isso resolve menos instâncias (4/10 médias). A vantagem do IDA* é a memória: nas médias usou em média 0,50 MB, contra 2,53 MB do A*, apesar de também guardar a tabela de uma iteração.

Recomendação. Para este problema recomendamos o A* em grafo com h4 (igual a h3 em movimentos): é óptimo nos dois critérios, é o que resolve mais instâncias e expande menos nós entre os algoritmos óptimos. O IDA*+TT só se justifica se a memória for o limite (tabuleiros 8×8 muito abertos) e com o critério movimentos. A BFS chega para minimizar movimentos

em instâncias até dezenas de milhares de estados. A Gulosa é útil quando basta uma solução rápida, sem garantia de óptimo.

Os dois critérios (exigência específica 1). Na base, os óptimos coincidem: 8 movimentos e custo 40, e a solução de mínimo de movimentos (A+1 C-3 O+3 P-1 B-1 R-2 Q+2 X+3) também tem custo ponderado 40. Coincidem porque a solução óptima move cada veículo uma única vez, pelo deslocamento mínimo necessário, e na base não há forma de poupar casas acrescentando movimentos. Com o gerador encontrámos instâncias em que os critérios diferem de facto. Para não depender de desempates, calculámos sobre o grafo completo o menor custo ponderado entre TODAS as soluções de mínimo de movimentos (`heuristicas.divergencia`):

Tabela 10 – Instâncias em que os dois critérios de optimização divergem

Instância	Mín. movimentos	Melhor custo ponderado com mín. movimentos	Óptimo ponderado	Movimentos do óptimo ponderado
pequena_02	7	24	21	8
media_07	17	65	64	18

Em 2 das 21 instâncias analisadas (base, pequenas e médias), nenhuma solução com o número mínimo de movimentos é óptima no custo ponderado: por exemplo, na `pequena_02` o mínimo são 7 movimentos, mas o melhor dessas soluções custa 24, enquanto o óptimo ponderado, 21, precisa de 8 movimentos (mover um carro em vez de um camião, ou deslocar um veículo menos casas com um movimento extra). Assim, a BFS (óptima em movimentos) deixa de ser óptima no custo ponderado, como se vê na coluna Custo/óptimo da BFS nas tabelas do critério ponderado.

6.4 Resposta à pergunta de análise

Qual é o número de estados alcançáveis na instância base, e que fracção deles o A com h2 expande?*

Uma BFS exaustiva a partir de s0 (função `espaco_alcancavel`) encontra **1079 estados alcançáveis** na base (4 deles objectivos e 11 714 transições), num limite teórico de 160 000 configurações sem colisões. O A* com h2 expande **357 nós no critério movimentos, isto é, 33,1 % dos estados alcançáveis**, contra 783 (72,6 %) com h1, 530 (49,1 %) com h3/h4 e 1057 (98,0 %) com h0/UCS. No critério ponderado, o A* com h2 expande 1069 nós (99,1 %): h2 conta veículos e é demasiado pequena face a custos de 2 a 9 por acção, enquanto h4 reduz para 816 (75,6 %). Os valores são verificados pelos testes (`test_hstar_base`) e reproduzem-se com `python experiencias.py` (`outputs/analise_base.txt`). Em ambos os casos o custo encontrado é o óptimo (8 movimentos; custo 40).

A fracção é alta porque o espaço da base é pequeno e muito ligado: a solução óptima exige mover 7 dos 8 veículos, logo grande parte dos estados tem $f \leq C^*$. A h2 expande menos do que h3/h4 em movimentos porque é, em média, maior, mas o preço é a perda da garantia de optimalidade (contra-exemplo da Secção 4.2).

7 CONCLUSÕES

7.1 Síntese

Retomando os objectivos específicos:

1. **Formular o problema** — atingido. O estado é um tuplo imutável de posições (suficiente e mínimo), as acções são deslocamentos de k casas e há dois custos (Capítulo 3). A base tem 1079 estados alcançáveis e óptimos de 8 movimentos e custo 40.
2. **Implementar os sete algoritmos** — atingido. BFS, DFS, IDS, UCS, Gulosa, A* e IDA* com tabela de transposição sobre uma interface genérica, verificados pelos testes automáticos (68 passed, 2 skipped).
3. **Heurísticas e provas** — atingido. h1, h3 e h4 são admissíveis e consistentes (provas e verificação exaustiva); a dica h2 é inadmissível em movimentos (contra-exemplo $h2 = 4 > h^* = 3$) e admissível no ponderado ($h2 \leq h4$).
4. **Gerador reprodutível** — atingido. 30 instâncias com a semente 202606 e óptimo certificado, ≥ 15 movimentos nas médias e grandes, incluindo instâncias onde os dois critérios divergem.
5. **Comparação experimental** — atingido. Os resultados confirmam a teoria: completude e optimalidade dos algoritmos óptimos, explosão de DFS/IDS em árvore, vantagem do A*+h4 e redução de nós do IDA* pela tabela de transposição.

7.2 Limitações

- Nas instâncias pequenas o mínimo de 15 movimentos foi substituído pelo maior óptimo que DFS e IDS resolvem (6 a 8 movimentos), porque as duas exigências são incompatíveis com DFS/IDS de controlo de ciclos só no caminho (Secção 5.4).
- Os tempos foram medidos com `tracemalloc` activo e com 2 a 3 execuções em paralelo, num portátil; os valores absolutos são indicativos, mas as contagens de nós são exactas e determinísticas.
- A implementação é em Python puro (≈ 10^4 expansões/s); com o limite de 60 s, isso define a fronteira entre resolvido e falhou nas instâncias grandes.

- A h2 do enunciado não é admissível em movimentos; os resultados com h2 nesse critério não têm garantia de óptimo, e numa instância grande (grande_05) o resultado não foi óptimo.
- As instâncias grandes (290 000 a 780 000 estados) não chegaram para esgotar o tempo da BFS e do UCS (só DFS, IDS e IDA* falharam): o gerador certifica o óptimo enumerando toda a componente, o que limita as grandes a menos de 10^6 estados com a memória disponível (cerca de 3 GB).

7.3 Trabalho futuro

- Heurística por base de dados de padrões (pattern database) para o grupo de veículos à frente de X, que capture interacções entre bloqueadores, com prova de admissibilidade.
- Representação em bits (bitboards) e geração incremental de acções para multiplicar a velocidade por 10–50 e resolver componentes de milhões de estados.
- Busca bidireccional e IDA* com tabela de transposição persistente entre iterações (com limites guardados) para reduzir as re-expansões no critério ponderado.

8 REFERÊNCIAS BIBLIOGRÁFICAS

Collette, S., Raskin, J.-F., & Servais, F. (2006). On the symbolic computation of the hardest configurations of the Rush Hour game. In *Computers and Games* (Lecture Notes in Computer Science, Vol. 4630, pp. 220–233). Springer.

Culberson, J. C., & Schaeffer, J. (1998). Pattern databases. *Computational Intelligence, 14*(3), 318–334.

Dijkstra, E. W. (1959). A note on two problems in connexion with graphs. *Numerische Mathematik, 1*(1), 269–271.

Flake, G. W., & Baum, E. B. (2002). Rush Hour is PSPACE-complete, or "Why you should generously tip parking lot attendants". *Theoretical Computer Science, 270*(1–2), 895–911.

Hart, P. E., Nilsson, N. J., & Raphael, B. (1968). A formal basis for the heuristic determination of minimum cost paths. *IEEE Transactions on Systems Science and Cybernetics, 4*(2), 100–107.

Korf, R. E. (1985). Depth-first iterative-deepening: An optimal admissible tree search. *Artificial Intelligence, 27*(1), 97–109.

Reinefeld, A., & Marsland, T. A. (1994). Enhanced iterative-deepening search. *IEEE Transactions on Pattern Analysis and Machine Intelligence, 16*(7), 701–710.

Russell, S., & Norvig, P. (2021). *Artificial Intelligence: A Modern Approach* (4.ª ed.). Pearson.

Universidade Eduardo Mondlane, Faculdade de Engenharia. (2026). *Projecto Prático 2026 — Resolução de Problemas de Busca em Python* (enunciado de Inteligência Artificial I).

9 ANEXO A — DECLARAÇÃO DE USO DE FERRAMENTAS DE IA

O grupo usou assistentes de IA como apoio, conforme a secção 5.4 do enunciado. A tabela regista as ferramentas, para que foram usadas e o que o grupo verificou. **[A REVER PELO GRUPO antes da entrega: confirmar e completar com o uso de IA de cada membro.]**

Tabela 11 – Uso de ferramentas de IA

Ferramenta	Para quê	Pedido principal (resumo)	O que o grupo verificou ou alterou	Secção / ficheiro
Assistente de IA (usado na preparação do domínio)	Código e testes do domínio	Implementar o contrato do domínio P6 (estado, acções, custos) e testes	Convenções discutidas em grupo; testes executados	dominio.py, problema.py, tests/test_dominio.py
Claude Code (Anthropic, modelo Claude Opus)	Código, depuração e revisão	Implementar as buscas, heurísticas h1–h4 com provas, gerador com semente 202606, experiências, menu e testes, a partir do enunciado e do contrato do domínio	Execução de todos os testes; verificação exaustiva h ≤ h*; conferência manual dos sucessores de s0 e do contra-exemplo de h2; nova execução das experiências	buscas.py, heuristicas.py, gerador.py, experiencias.py, main.py, tests/
Claude Code (Anthropic, modelo Claude Opus)	Texto do relatório	Redigir o relatório no template oficial com os números reais dos CSV	Revisão do texto, das provas e das conclusões pelos membros	Relatório (todos os capítulos)

O grupo declara que compreende e consegue explicar todo o código e texto entregues, e que os resultados experimentais foram obtidos pela execução do código com a semente 202606 e são reproduzíveis com `python experiencias.py`.

Assinaturas: _____ _____

_____ _____ _____

10 ANEXO B — CONTRIBUIÇÃO DOS MEMBROS

Tarefas segundo o quadro do sprint (S01–S35). **[A CONFIRMAR PELO GRUPO: percentagens e tarefas efectivamente realizadas; o histórico Git é entregue com o projecto.]**

Tabela 12 – Contribuição individual

Membro (Apelido, Nome)	Tarefas principais	Ficheiros / secções	Contribuição (%)	Assinatura
Falaque, Dalton	S05, S11, S12, S18, S29: interface das buscas; BFS, UCS, DFS, IDS; garantias	buscas.py; Cap. 2 e 6		
Nhabinde, Alex	S06, S13, S17, S24, S30: heurísticas h1–h4, provas, Gulosa e A*; integração	heuristicas.py, buscas.py; Cap. 4		
Nhabomba, Lino	S02, S03, S09, S10, S16, S31: contrato, estado, domínio e testes do domínio	dominio.py, problema.py, tests/test_dominio.py; Cap. 3		
Quive, Alexandre	S07, S14, S19, S22, S23, S32: IDA* e tabela	buscas.py (IDA*), gerador.py; Secções 2.4		

	de transposição; gerador	e 5.4		
Ricardo	S04, S08, S15, S20, S25, S27, S28, S33: repositório, métricas, menu, experiências, montagem	main.py, experiencias.py; Cap. 5–6		

11 ANEXO C — CÓDIGO-FONTE RELEVANTE

O código completo está no repositório (ficheiro ZIP entregue, com a pasta `.git`). Seguem os excertos centrais: a interface genérica e o IDA* com tabela de transposição.

```
"""Contrato genérico para algoritmos de busca."""
from abc import ABC, abstractmethod
from dataclasses import dataclass
from typing import Any



class Problema(ABC):
    @abstractmethod
    def estado_inicial(self): ...

    @abstractmethod
    def accoes(self, s): ...

    @abstractmethod
    def resultado(self, s, a): ...

    @abstractmethod
    def custo(self, s, a, s2): ...

    @abstractmethod
    def e_objectivo(self, s): ...

    @abstractmethod
    def heuristica(self, s, nome): ...



@dataclass
class No:
    estado: Any
    pai: "No | None" = None
    accao: Any = None
    g: float = 0
    profundidade: int = 0



def caminho(no):
    """Lista de nós da raiz ao destino, incluindo ambos."""
    nos = []
    while no is not None:
        nos.append(no)
        no = no.pai
    return list(reversed(nos))


def idastar(problema, heuristica="h1", tempo_max=None, tabela=True, max_iter=10_000, max_nos=None,
**_):
    """IDA*: DFS com limite em f = g + h, que sobe para o menor f que o excedeu.

    Com `tabela=True` usa uma tabela de transposição estado -> menor g visto
    na iteração actual (limpa em cada iteração). Um estado reencontrado com
    g >= ao guardado é podado: a subárvore já foi explorada com pelo menos a
    mesma folga (limite - g), logo não pode conter uma solução nova dentro do
    limite. Sem tabela só há controlo de ciclos no caminho actual."""
```

```
    r = Resultado("IDA*+TT" if tabela else "IDA*")
    rel = _Relogio(tempo_max, max_nos)
    expandidos_antes = set()
    r.extra.update(heuristica=heuristica, podas_tabela=0)
    INF = float("inf")


    def h(s):
        return problema.heuristica(s, heuristica)


    def correr():
        raiz = No(problema.estado_inicial())
        r.gerados = 1
        limite = h(raiz.estado)
        no_caminho = set()
        tt = {}


        def rec(no, hn):
            rel.verificar()
            f = no.g + hn
            if f > limite:
                return None, f
            if problema.e_objectivo(no.estado):
                return no, f
            if tabela:
                visto = tt.get(no.estado)
                if visto is not None and visto <= no.g:
                    r.extra["podas_tabela"] += 1
                    return None, INF
                tt[no.estado] = no.g
            r.expandidos += 1
            if no.estado in expandidos_antes:
                r.reexpandidos += 1
            else:
                expandidos_antes.add(no.estado)
            no_caminho.add(no.estado)
            r.fronteira_max = max(r.fronteira_max, len(no_caminho) + 1)
            minimo = INF
            sucessores = []
            for a in problema.accoes(no.estado):
                filho = _filho(problema, no, a)
                r.gerados += 1
                if filho.estado in no_caminho:
                    continue
                sucessores.append((filho.g + h(filho.estado), filho))
            sucessores.sort(key=lambda t: t[0])       # ordenar filhos por f
            for fh, filho in sucessores:
                achado, novo = rec(filho, fh - filho.g)
                if achado:
                    return achado, novo
                minimo = min(minimo, novo)
            no_caminho.discard(no.estado)
            return None, minimo


        while r.iteracoes < max_iter:
            r.iteracoes += 1
            tt.clear()
            no_caminho.clear()
            achado, novo = rec(raiz, limite)
            if achado:
                r.estado, r.no = "ok", achado
                return
            if novo == INF:
                r.estado = "sem solução"
                return
            limite = novo
        r.estado = "limite"
    return _executar(r, correr)


def h4(p, s):
    b = bloqueadores(p, s)
    if b is None:
        return INF
    d = p.tamanho - 2 - s[p.indice("X")]
    ponderado = p.criterio == "ponderado"
    total = (2 * d if ponderado else (1 if d > 0 else 0))
    for vid in b:
        opcoes = saidas(p, s, vid)
```

```
    if not opcoes:
        return INF
    L = p.veiculos[p.indice(vid)].comprimento
    total += min(L * abs(k) for k, _ in opcoes) if ponderado else 1
return total
```